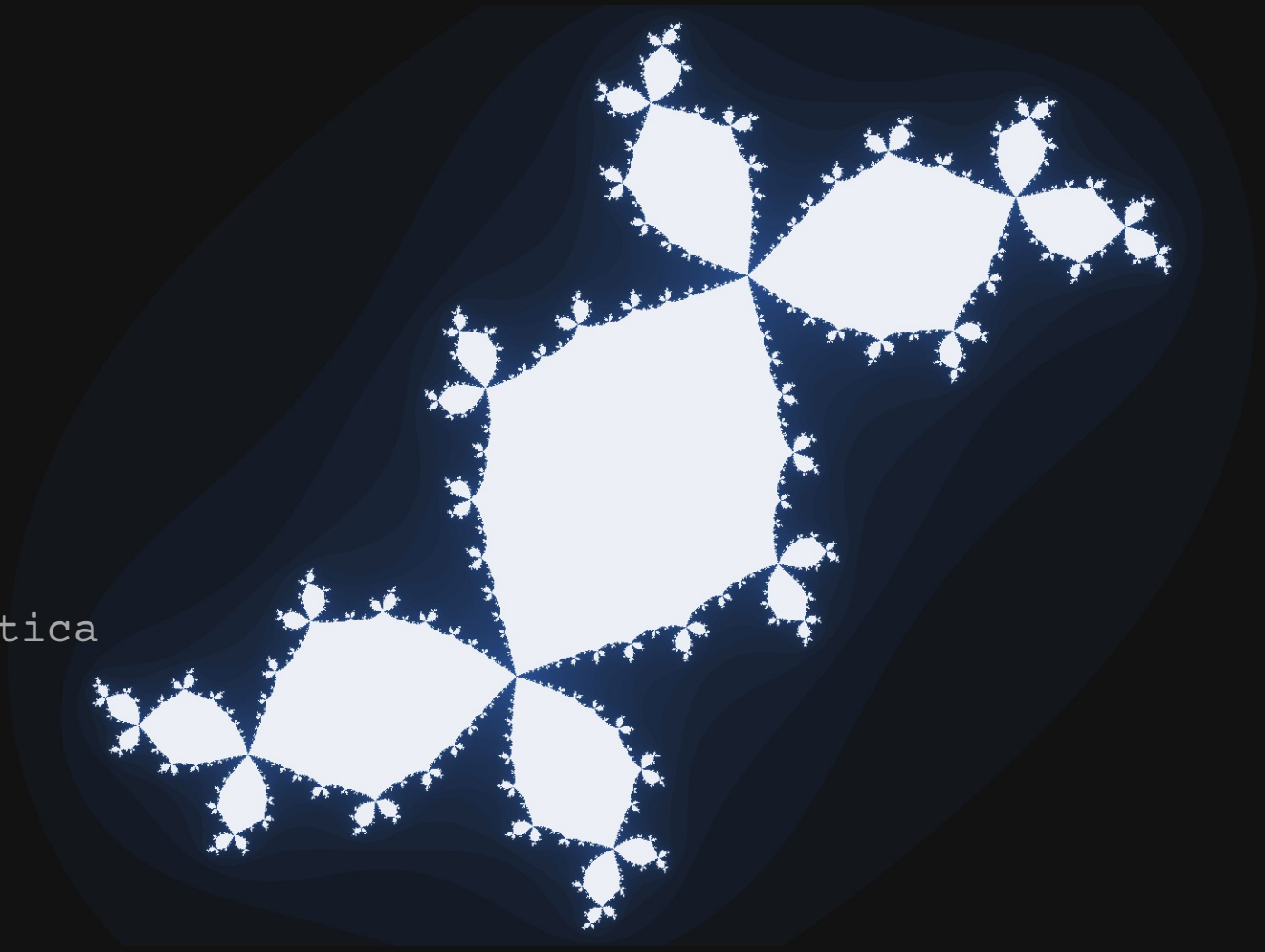
Julia × MPI

mestre/escravo no cluster Atlântica



Francisco Freitas · Pedro Haggstram

O fractal

cada pixel é independente, mas não custa o mesmo

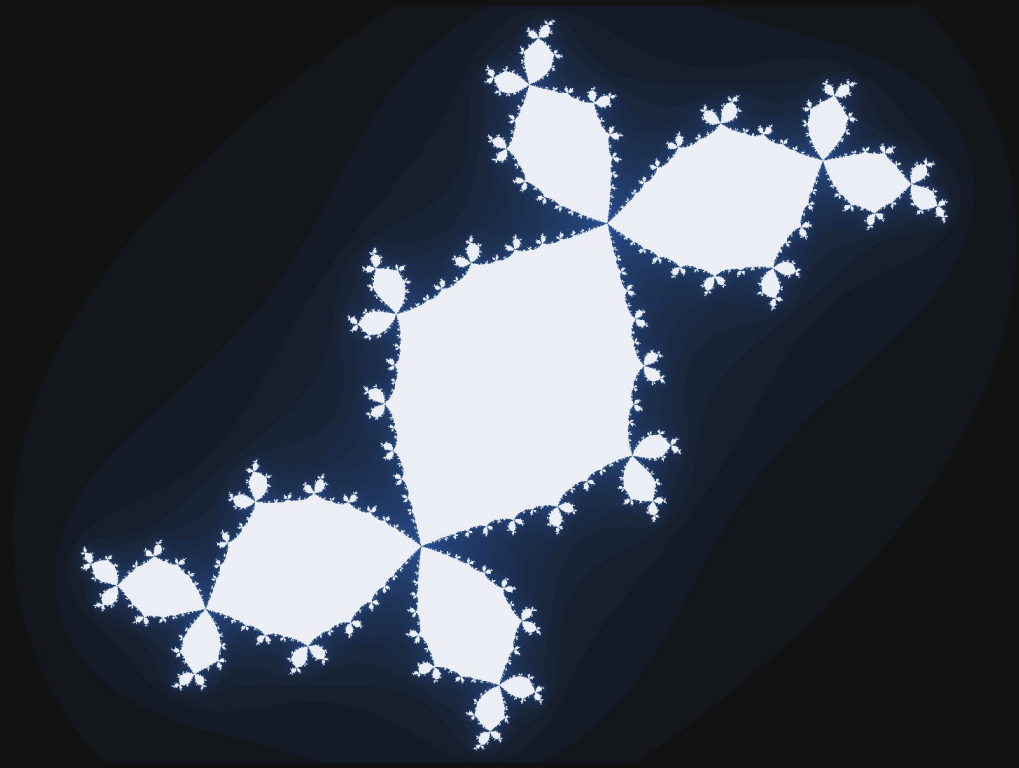
$$z = z^2 + c$$

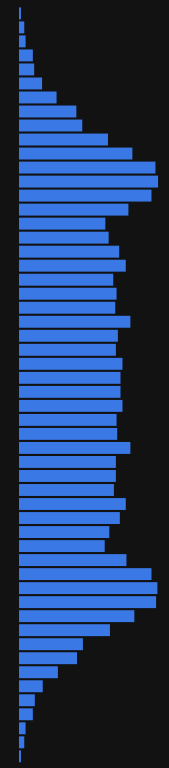
$c = -0{,}123 + 0{,}745i$

pixel = iterações até $|z| > 2$

interior = max_iter (6000)

borda = quase nada

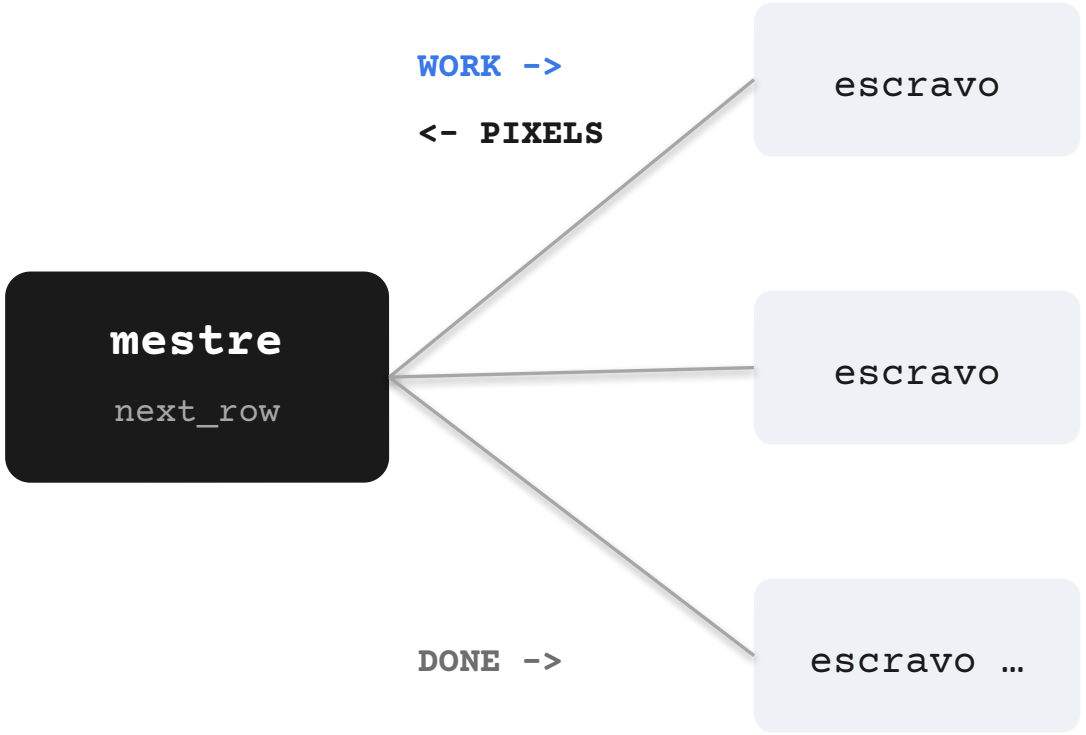




custo por linha

Mestre/escravo

tarefa = bloco de 4 linhas · saco = índice da próxima linha livre

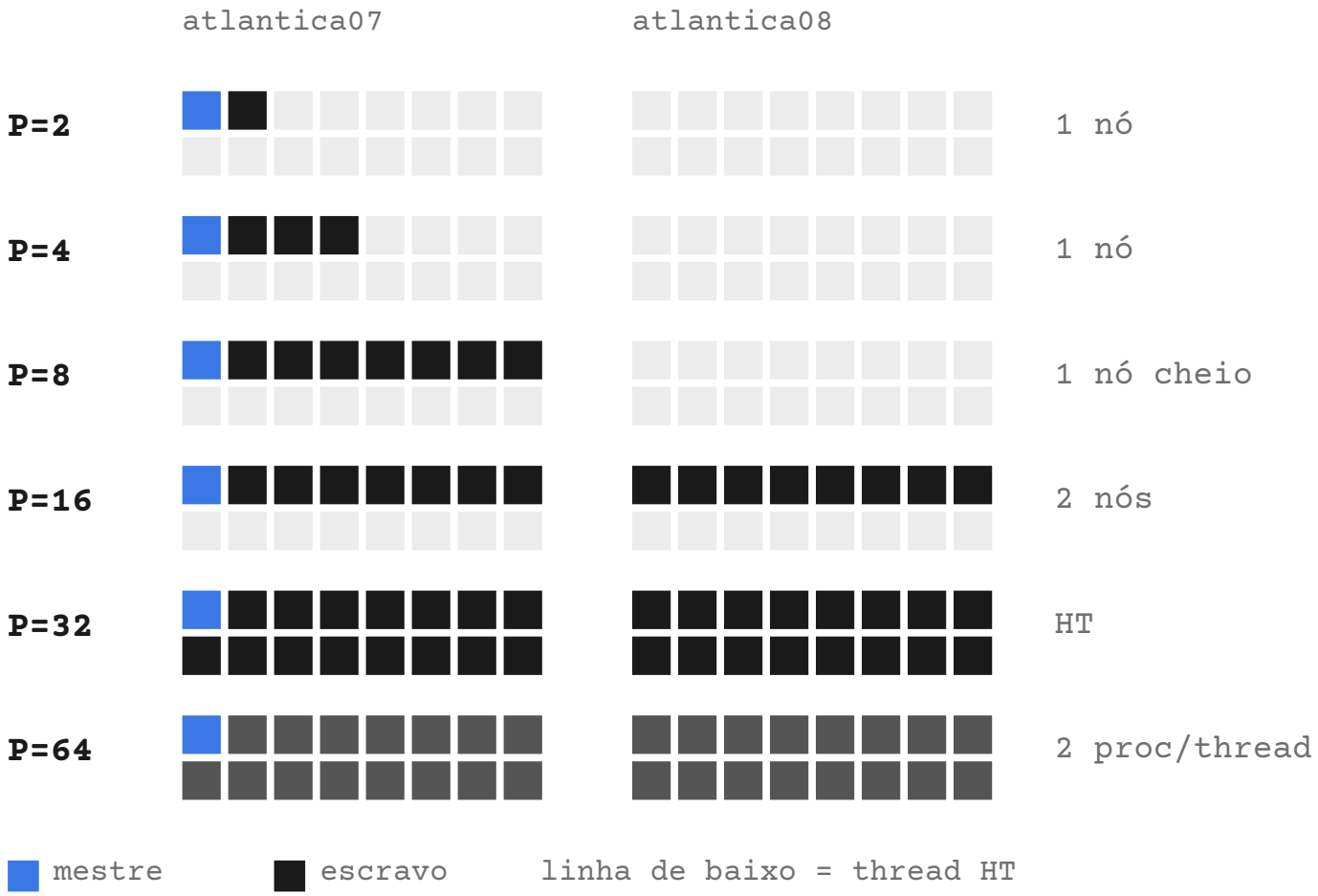



```
mestre:
  while escravos ativos:
    recv(ANY_SOURCE)
    copia linhas p/ imagem
    if sobra: send WORK(row, 4)
    else:     send DONE

escravo:
  send REQUEST
  while recv() == WORK:
    calcula 4 linhas
    send PIXELS
```

Como rodamos

cluster Atlântica (LAD) · 2 nós exclusivos · sbatch



3840×2160

max_iter 6000

T1 = 40,4 s

sequencial, mesmo nó

10× cada

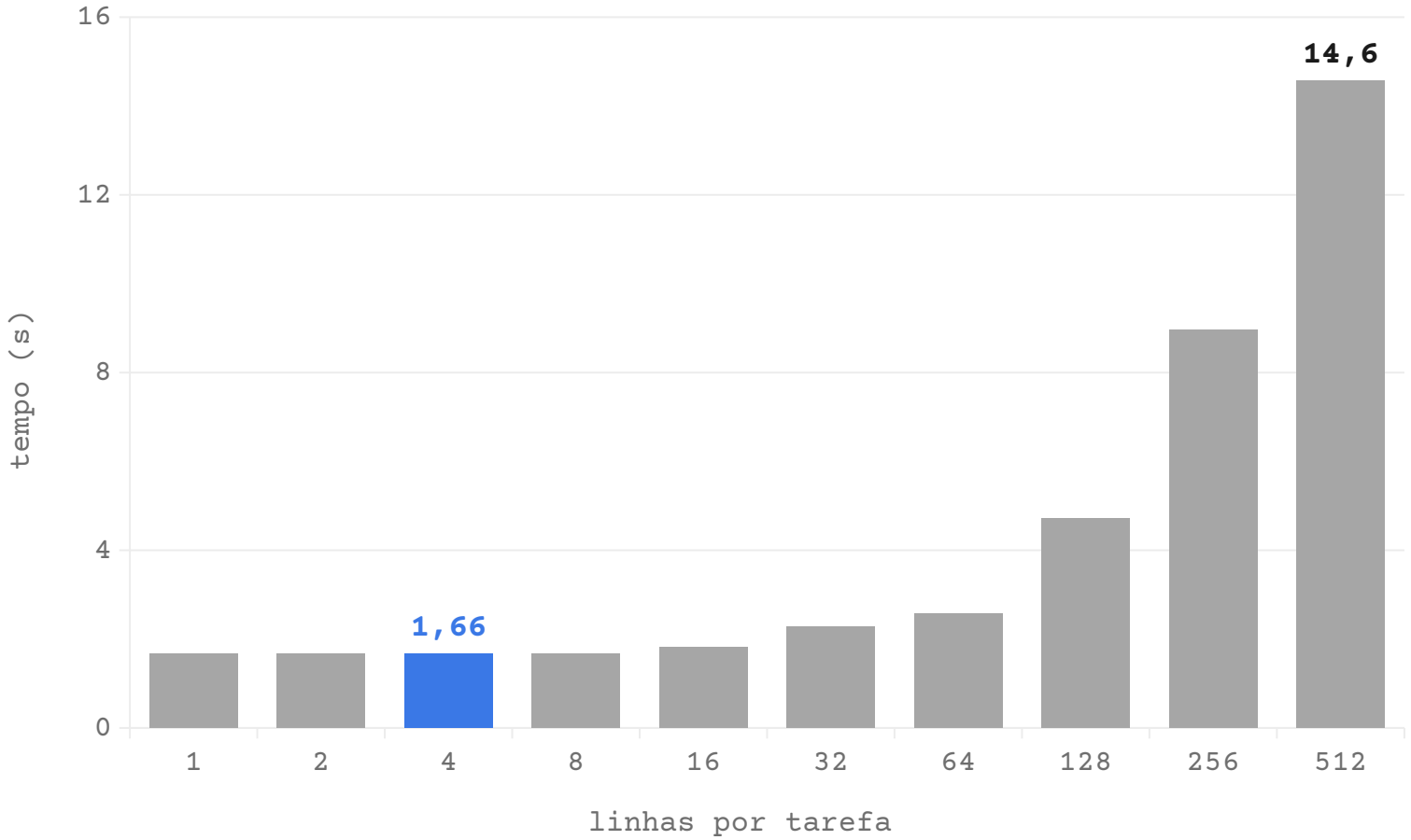
mediana · min/max < 1%

checksum

igual ao sequencial

Granularidade

P = 32 · de 1 até 512 linhas por tarefa



4 linhas

540 tarefas · escolhida

94 ms

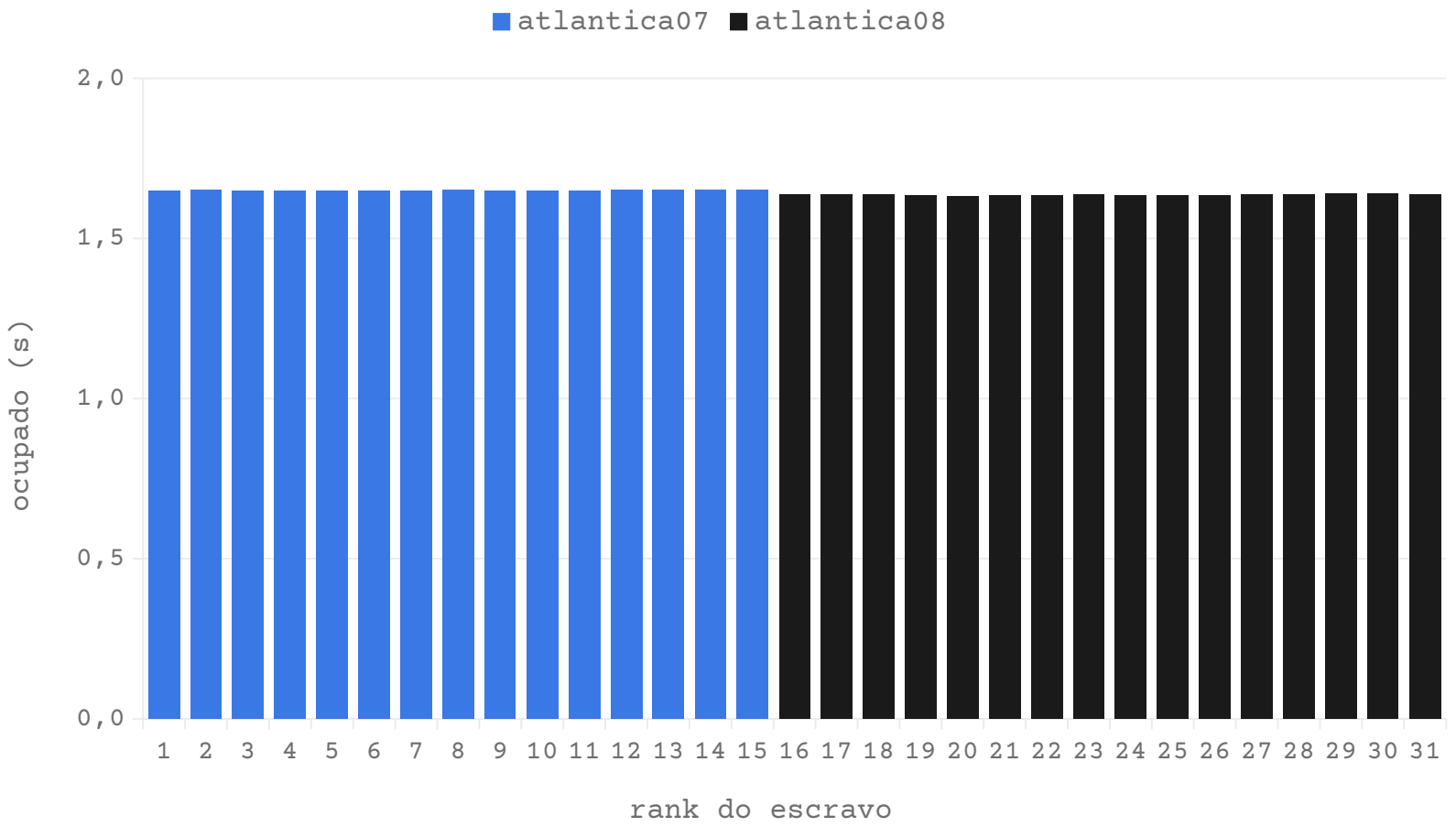
cálculo de uma tarefa

0,65 ms

ida e volta entre nós

Balanceamento

P = 32 · 4 linhas por tarefa · tempo ocupado de cada escravo



99%

do tempo calculando

± 2 ms

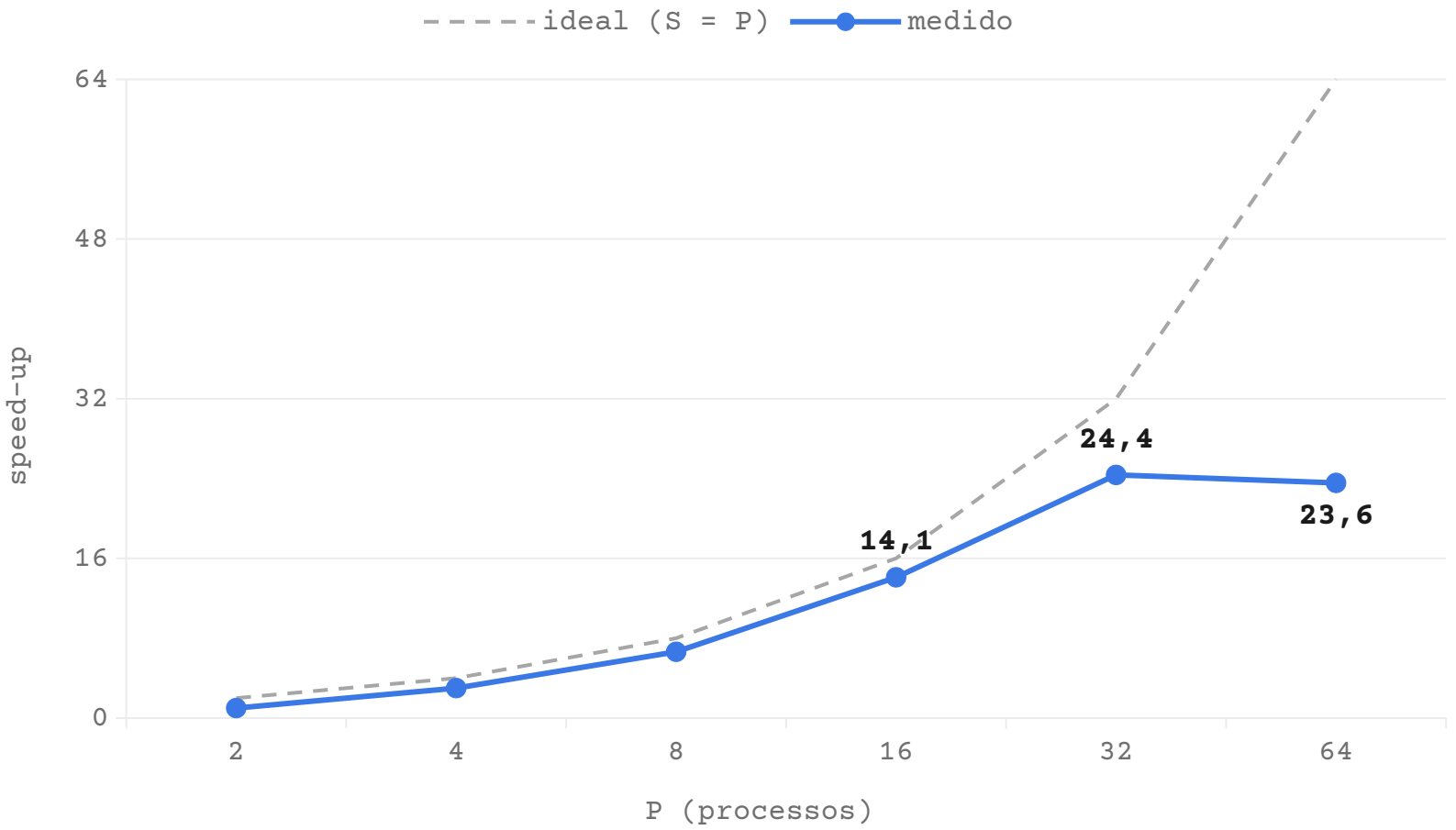
liberados juntos (TAG_DONE)

16–20

tarefas por escravo

Escalabilidade forte

mesma imagem 4K, P = 2 … 64 · S = T1 / TP



1,66 s

com P = 32 (T1 = 40,4 s)

×1,26

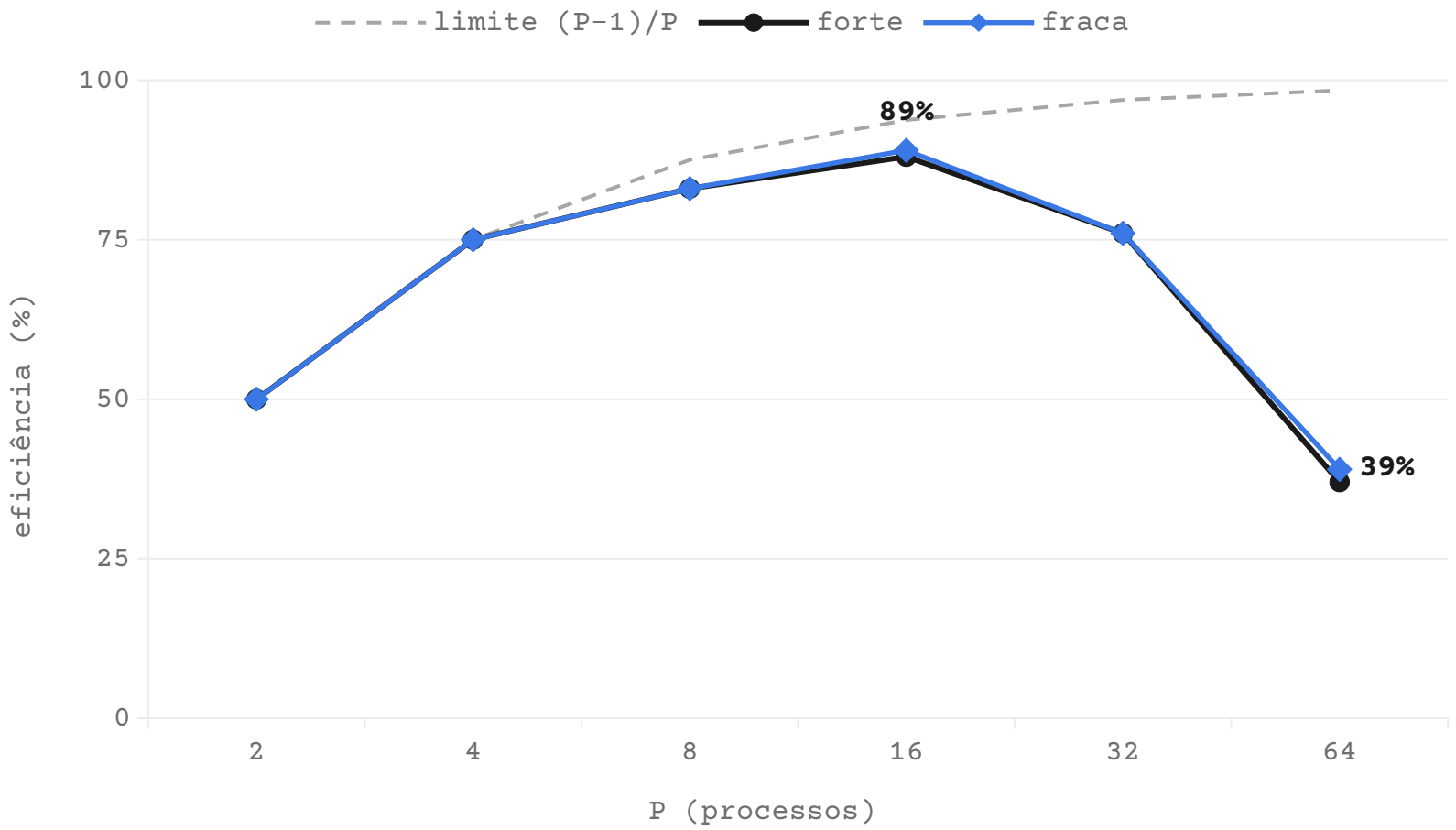
lentidão por núcleo com HT

P = 64

sem ganho: 2 proc/thread

Forte × fraca

fraca: altura = 2160·P (trabalho cresce com P)



≈ iguais

curvas a menos de 2%

2× … 64×

trabalho total

46–105 s

por execução

Conclusão

24×

speed-up com 32 processos

99%

dos escravos calculando

100%

checksums = sequencial

limite: **núcleos físicos, HT e o núcleo do mestre**

não: **comunicação nem balanceamento**

github.com/fcmfreitas/c_mpi